



INDIA.RUNS

Build what next India runs on

Team Name : Solo — Jashwanth

Team Leader Name : Jashwanth Sangu

Problem Statement : Intelligent Candidate Discovery & Ranking Challenge

Solution Overview

What is your proposed solution?

A multi-signal, embedding-augmented candidate ranking system that combines dense semantic retrieval with 76 hand-engineered features to rank 100K candidates for a Senior AI Engineer role.

Pipeline uses BAAI/bge-large-en-v1.5 embeddings for semantic matching, cross-encoder (ms-marco-MiniLM-L-6-v2) re-ranking of top-1000, and extracts career-depth, skill-quality, and behavioral signals from structured candidate data.

Applies a multiplicative scoring formula: $\text{score} = \text{relevance_gate} \times \text{technical_core} \times \text{fit_multiplier} \times \text{behavioral} \times \text{availability}$. XGBoost LTR (rank:pairwise) trained on pseudo-relevance labels, blended 60/40 with heuristic scores.

Produces ranked top-100 with natural-language reasoning grounded entirely in candidate data — no LLM used for scoring or reasoning generation, eliminating hallucination risk.

What differentiates your approach from traditional candidate matching systems?

1. Career-depth analysis over keyword matching: We analyze career trajectories — months in retrieval/ranking roles, shipped production ML systems, skill-career coherence — not just skill keyword counts.
2. Multiplicative gating architecture: A relevance gate (title + career + coherence) acts as a hard filter — candidates with irrelevant titles or stuffed profiles get multiplied to near-zero regardless of other signals.
3. 8-rule honeypot detection: Purpose-built detectors for impossible timelines, expert-zero-usage claims, inverted salaries (catches ~19% of dataset), assessment-proficiency gaps, and template summaries with career mismatches.
4. Zero-hallucination reasoning: Every reasoning sentence is template-generated from extracted features with 5 lead rotation variants and JD phrase

JD Understanding & Candidate Evaluation

Key requirements extracted from the JD:

Tier-1 (Core) Skills: Information Retrieval, Search/Ranking Systems, NLP/NLU, Recommendation Systems, Transformers/BERT, Vector Databases, Embedding Models, LLM/RAG — 27 skills weighted 2x in skill matching.

Tier-2 (Adjacent) Skills: Deep Learning, PyTorch/TensorFlow, MLOps, Data Engineering, Distributed Systems, Python, Computer Vision — 26 standard ML engineering skills.

Experience: 5-12 year sweet spot (Gaussian centered at 7, $\sigma=2.0$); career history in retrieval/ranking/search at product companies (not consulting firms).

Location: India-based preferred, with bonus for Bangalore, Hyderabad, Mumbai, Delhi/NCR, Pune, Chennai, Noida.

How does your solution evaluate candidate fit beyond keyword matching?

Title Relevance Tiers (0-4): T4 = ML/AI/NLP/Search Engineer (1.0x), T3 = SWE/Backend/Data (0.6x), T2 = DevOps/Frontend (0.25x), T1 = Manager (0.08x), T0 = Non-tech (0.02x). Each tier is a multiplier, not a filter.

Career Depth Scoring: Months spent in AI/ML roles, months in retrieval/ranking specifically, number of production ML deployments shipped, and whether career was at product companies vs. consulting.

Skill-Career Coherence: Cross-validates that listed AI skills actually appear in career history. 'PyTorch, Transformers, RAG' listed but career entirely in manual testing → coherence near 0 → keyword stuffer penalty (x0.05).

Behavioral Signals: Open-to-work status, recruiter response rate, GitHub activity score, notice period, days since active, platform assessment scores — these inform availability and engagement, not core ranking.

Ranking Methodology

How does your system retrieve, score, and rank candidates?

Step 1 — Semantic Retrieval: All 100K candidate profiles embedded using BAAI/bge-large-en-v1.5 (1024-dim). JD embedded separately. Cosine similarity (dot product on L2-normalized vectors) provides dense retrieval score.

Step 2 — Cross-Encoder Re-rank: Top-1000 by BM25+embedding are re-ranked with cross-encoder/ms-marco-MiniLM-L-6-v2 for fine-grained relevance scoring during precompute.

Step 3 — Feature Extraction: 76 features extracted per candidate covering title relevance, career depth, skill matching, experience fit, behavioral signals, and 8 honeypot indicators.

Step 4 — Scoring: $\text{final_score} = \text{relevance_gate} \times \text{technical_core} \times \text{fit_multiplier} \times \text{behavioral} \times \text{availability}$. $\text{Relevance gate} = 0.35 \times \text{title} + 0.45 \times \text{career} + 0.20 \times \text{coherence}$. $\text{Technical core} = 0.35 \times \text{retrieval} + 0.25 \times \text{production} + 0.15 \times \text{vector_db} + 0.10 \times \text{shipped} + 0.15 \times \text{semantic}$.

What models, algorithms, or heuristics are used?

Model: BAAI/bge-large-en-v1.5 (sentence-transformers) — 1024-dim dense embeddings, top-tier MTEB retrieval benchmarks, L2 normalization. Cross-encoder: ms-marco-MiniLM-L-6-v2 for passage re-ranking.

XGBoost LTR: rank:pairwise objective, 78 features, Optuna-tuned hyperparams (50 trials). Pseudo-labels (0-4 graded relevance). Final = $0.6 \times \text{XGBoost} + 0.4 \times \text{heuristic blend}$ for score discrimination.

Heuristics: Title tier classification, career trajectory analysis, honeypot detection (8 rules), hard penalty multipliers for consulting-only (x0.10), keyword stuffers (x0.05), non-tech titles (x0.20), CV-only specialists (x0.40).

Explainability & Data Validation

How are ranking decisions explained?

Each candidate receives a 3-part natural-language reasoning: (1) Lead sentence — career-specific opener using the candidate's most distinctive signal (notable company, tier-1 education, retrieval depth, shipped systems, or ML certifications).

(2) JD connection phrase — links a specific signal to a JD requirement, rotated across 10+ applicable phrases using candidate_id hash for diversity.

(3) Supporting signals and honest concerns (notice period, responsiveness, consulting background).

5 lead rotation variants prevent templated reasoning. Rank-tier prefixes based on candidate signals, not just rank position — a strong candidate at rank 91 (due to tie-breaking) won't get 'borderline fit' if they have 12+ months retrieval at a product company.

How do you prevent hallucinations or unsupported justifications?

No LLM is used for reasoning generation. Every sentence references extracted feature values directly. If career_retrieval_months=36, reasoning says '3+ years in ranking and retrieval' — a direct computation, not a generation. Zero hallucination risk.

All numerical claims (response rates, GitHub scores, notice periods, skill counts) are extracted from structured candidate data fields and directly formatted into strings.

How does your solution handle inconsistent, low-quality, or suspicious profiles?

Honeypot Detection (8 rules): impossible timelines, expert-zero-usage (>3 skills), salary min>max (catches ~19%), assessment-proficiency gaps, template summaries with career mismatches, suspicious endorsement ratios, ghost profiles.

Hard Penalties: consulting-only careers (x0.10), keyword stuffers with low coherence (x0.05), non-tech titles (x0.20). These multiplicative penalties

End-to-End Workflow

Complete workflow from JD input to ranked candidate output:

Step 1 — JD Parsing: Extract tier-1 (core retrieval/ranking) and tier-2 (adjacent ML) skills from job description. Define experience sweet spot (5-12 years), preferred locations, and title relevance tiers.

Step 2 — GPU Precompute (Colab/Kaggle): Load 100K candidates from candidates.jsonl. Embed all candidate profile texts + JD using BAAI/bge-large-en-v1.5 on GPU. Compute BM25 scores. Cross-encoder re-rank top-1000. Extract 76 features per candidate. Save embeddings.npy, jd_embedding.npy, and features.parquet to artifacts directory.

Step 3 — CPU Ranking (<5 min, no GPU): Load precomputed artifacts. Compute cosine similarity (dot product on L2-normalized embeddings). Train XGBoost LTR on pseudo-labels (rank:pairwise, 78 features). Score all 100K candidates using multiplicative formula. Blend 60% XGBoost + 40% heuristic.

Step 4 — Reasoning Generation: For each top-100 candidate, generate 3-part natural-language reasoning from extracted features. No LLM call — pure template logic with 5 rotation variants.

Step 5 — Output: Write jashwanth_s.csv with columns: candidate_id, rank, score, reasoning. Scores range from 1.0 (rank 1) to ~0.75 (rank 100).

Total Runtime: GPU precompute ~15 min (one-time) + CPU ranking <3 min = submission-ready in under 20 minutes.

System Architecture

Source Modules:

src/precompute.py — GPU orchestrator — embeddings, BM25, cross-encoder, feature extraction

src/features.py — 76-field CandidateFeatures dataclass + extraction (673 lines)

src/scoring.py — Multiplicative scoring engine — 5-stage score composition

src/reasoning.py — NL reasoning generation — 5 lead variants, JD phrase rotation

src/honeypot.py — 8-rule fraud/synthetic profile detection

src/rank.py — XGBoost LTR + heuristic blend, CPU ranking + output (681 lines)

run_pipeline.py — Single-command end-to-end pipeline execution

Architecture: Two-phase split

GPU Phase: candidates.jsonl → Embedding Engine (bge-large-en-v1.5, 1024-dim) + Feature Extraction (76 signals) + Cross-Encoder Re-rank → artifacts/ (embeddings.npy, features.parquet)

CPU Phase: Load artifacts → Cosine similarity + XGBoost LTR (rank:pairwise, 78 features) + Heuristic scoring → Multiplicative composition (relevance x quality x availability) → Top-100 + reasoning → jashwanth_s.csv

Results & Performance

Ranking quality insights:

Top-100 composition: 100% AI/ML engineers at product companies in India. #1 candidate: Senior ML Engineer with 3+ years in ranking/retrieval systems at product companies. Score range: 1.0 (rank 1) to ~0.75 (rank 100) — good discrimination throughout.

Title distribution: All top-100 candidates have title tier T3 or T4 (ML/AI/NLP/Search/SWE/Data Engineer). Zero non-tech titles, zero consulting-only profiles, zero keyword stuffers survived the ranking.

Reasoning quality: Every reasoning is unique, career-specific, and grounded in data. Top-10 candidates mention specific retrieval/ranking experience durations and tier-1 institution backgrounds. Lower-ranked candidates include honest concerns.

Honeypot filtering: Validated on synthetic test candidates — impossible timelines, zero-usage experts, inverted salaries, and skill stuffers are all correctly suppressed. 8 detection rules catch ~19K suspicious profiles out of 100K.

Runtime and compute constraints:

GPU precompute: ~15 minutes on Google Colab (T4 GPU) or Kaggle (2x T4). Embeds 100K candidate texts + feature extraction. One-time cost, artifacts cached.

CPU ranking: <3 minutes on standard CPU, no GPU required, no network access. Loads precomputed artifacts, scores 100K candidates, outputs top 100 with reasoning. Well within the 5-minute CPU constraint.

Memory: Peak ~4GB during embedding (model + batch). Ranking phase uses <1GB. Compatible with 16GB constraint.

Evaluation metric: Optimized for $0.50 * \text{NDCG}@10 + 0.30 * \text{NDCG}@50 + 0.15 * \text{MAP} + 0.05 * \text{P}@10$. Multiplicative gating ensures top-10 are extremely relevant (boosting $\text{NDCG}@10$), while quality scoring maintains depth through top-50.

Technologies Used

Technologies, frameworks, and tools:

Python 3.10+

Core language for entire pipeline — chosen for ML ecosystem maturity and rapid prototyping.

sentence-transformers + BAAI/bge-large-en-v1.5

Dense embedding model for semantic similarity. 1024-dim, top-tier MTEB retrieval benchmarks, built-in L2 normalization.

cross-encoder/ms-marco-MiniLM-L-6-v2

Passage re-ranking model for fine-grained relevance scoring of top-1000 candidates during precompute.

XGBoost (rank:pairwise)

Learning-to-rank with 78 features, Optuna-tuned hyperparameters. Pseudo-relevance labels (0-4 graded).

Optuna

Bayesian hyperparameter optimization — 50 trials for XGBoost tuning (learning rate, depth, regularization).

NumPy

Vectorized cosine similarity (dot product on L2-normalized embeddings). Processes 100K similarity scores in milliseconds.

Pandas + PyArrow (Parquet)

Feature storage. Parquet format for columnar compression, fast I/O, type preservation — 76 feature columns.

Submission Assets

Submission deliverables:

GitHub Repository: <https://github.com/JASHWANTHS07/india-runs>

Submission File: jashwanth_s.csv (100 rows: candidate_id, rank, score, reasoning)

Colab Notebook: redrob_pipeline.ipynb — full pipeline execution on Colab T4 GPU

Kaggle Notebook: redrob_kaggle.ipynb — dual T4 GPU variant with visible progress bars

Source Code: src/features.py, src/scoring.py, src/reasoning.py, src/honeypot.py, src/rank.py, src/precompute.py

Pipeline Runner: run_pipeline.py — single-command end-to-end execution

Test Suite: 42 tests (pytest) — feature extraction, honeypot detection, scoring, reasoning

AI Tools Declared: Claude (Anthropic) — architecture design, code review, scoring iteration. No LLM at runtime.



INDIA.RUNS

Build what next India runs on

THANK YOU